# MetaLabe Linguistic Fidelity Registry

## Draft Laboratory Note and Instrument Specification v0.1

**Status:** Draft laboratory note / work in progress / pre-certification specification
**Date:** 3 November 2026

**Attribution**
Lu Semita (EmergenceByDesign), in collaborative co-authorship with the ChatGPT family, Nexus and Orion, Claude family, and Grok within MetaLabe.

## Notice

This draft does not claim ownership, priority, discovery, or final authority over linguistics, legal interpretation, jurisprudence, formal semantics, proof theory, Lean, natural-language processing, etymology, grammar, or any other established field or result referenced here. Established works, legal authorities, dictionaries, style manuals, linguistic traditions, and formal systems retain their own provenance and attribution.

The contribution of this laboratory note lies in the stated construction: a provenance-preserving instrument for comparing linguistic records, exposing semantic drift, retaining competing parses and domain rules, and emitting formal proof obligations and Lean-ready finite receipts for later independent certification.

The instrument does **not** claim that Lean can certify what a natural-language statement "really means." It separates the natural-language-to-formal bridge from the formal consequences of an admitted bridge. The current browser instrument performs local deterministic checks and emits Lean-ready source; it does not run or impersonate the Lean kernel.

---

## Abstract

Language preserves relation by compression. A word, clause, quotation, legal holding, theorem statement, transcript, dictionary entry, or later paraphrase can carry enough structure for reuse without carrying every condition that generated the earlier meaning. This capacity is indispensable and pharmakonic: compression allows continuity, but the same compression can erase scope, agency, modality, definitions, provenance, exceptions, or competing parses. A later generation may inherit the result while forgetting the basis by which the result was warranted.

The **MetaLabe Linguistic Fidelity Registry (MLFR)** operationalizes an existing MetaLabe principle: preserve not only the mark, but the parser, relation, frame, history, path, residue, and reopen condition. The instrument compares two frozen linguistic artifacts without asserting that either artifact

exhausts its referent. It distinguishes lexical similarity from relational continuity and records what is preserved, forgotten, added, transformed, unresolved, or dependent upon a declared interpretive bridge.

The system is intentionally not a semantic oracle. Automatic parsing produces **candidate observations**. An analyst or downstream parser may promote selected observations into a declared constraint registry. The formal layer then compares those declared registries. A generated Lean receipt can certify finite relations among the declared atoms – for example, that a condition present in record A is absent from record B, or that two declared finite constraint sets differ. Such a receipt certifies the formalized registry relation, not the correctness of the natural-language interpretation that populated the registry.

---

## 1. Continuity with MetaLabe and Thoth's Tool

The prior Thoth's Tool specification treats a written artifact as a frozen state of a moving semantic process and requires the artifact to remain attached to source input, parser paths, relation claims, confidence/status, residue, version, reopen conditions, and cascade triggers. Its compact requirement is:

> mark + parser + relation + frame + history + path + residue + reopen condition

The later MetaLabe relation registry adds typed relations such as `DEFINED_USING`, `REQUIRES`, `PRESERVES`, `FORGETS`, `IMPLIES`, `PROJECTS_TO`, and `TRANSFORMS_TO`, while explicitly refusing to install ontology before relational commitments are recorded.

MLFR applies those principles to linguistic continuity.

A sentence is therefore not stored as:

```
text -> meaning
```

but as:

```
artifact
  + provenance
  + domain
  + authority
  + parser observations
  + admitted constraints
  + competing parses
  + relation paths
  + residue
  + reopen conditions
```

---

## 2. The motivating failure mode

A later record can look similar while changing the relation it carries.

Example:

```
A: No employee shall knowingly disclose confidential information
   to an unauthorized third party.

B: No employee shall disclose information to a third party.
```

A similarity engine may correctly report extensive lexical overlap. A fidelity registry instead asks what changed:

- `knowingly` disappears;
- `confidential` disappears;
- `unauthorized` disappears;
- the prohibited information class broadens;
- the prohibited recipient class broadens;
- the mental-state condition is no longer carried.

The important output is not a scalar similarity score. It is a typed delta.

---

## 3. Frozen artifact model

For artifact `A`, let:

```
Artifact(A) =
  (raw_text,
   provenance,
   frame,
   domain,
   authority,
   observations,
   declared_constraints,
   parse_set,
   residue,
   reopen_conditions)
```

No component is declared metaphysically final. `raw_text` is itself a recording. `observations` are parser outputs. `declared_constraints` are the propositions admitted for the present comparison.

The distinction between observation and declaration is mandatory:

```
parser notices X
        !=
X has been formally admitted as the meaning of the source
```

---

## 4. Candidate parser observations

The v0.1 browser instrument performs deterministic, inspectable observations over English text. It marks candidate occurrences of:

- negation;
- universal/existential quantifiers;
- deontic and modal terms (`shall`, `must`, `may`, `should`, `can`, `will`);
- mental-state terms (`knowingly`, `intentionally`, `recklessly`, etc.);
- exception/condition terms (`unless`, `except`, `only if`, `provided that`);
- passive constructions using a deliberately simple auxiliary-plus-participle heuristic;
- explicit agent phrases introduced by `by`;
- lexical additions/removals after normalization.

These are observations, not adjudications.

The v0.1 parser deliberately prefers visible limitations over fluent overclaim. A missed passive construction is a parser limitation. An ambiguous `may` remains an observation requiring domain interpretation. The browser does not silently convert a heuristic hit into legal meaning.

---

## 5. Declared constraint registry

The analyst may declare constraints using one line per atom:

```
constraint_id :: relation_type :: expression
```

Example:

```
mental_state :: REQUIRES :: knowingly
information_scope :: REQUIRES :: confidential
recipient_scope :: REQUIRES :: unauthorized_third_party
prohibition :: IMPLIES :: no_disclosure
```

The identifier is a local handle, not an eternal semantic definition. The relation type identifies how the expression participates in the current registry.

For two artifacts A and B, the comparison partitions declared constraints into:

```
PRESERVED = constraints appearing identically in A and B
LOST      = constraints in A but not B
ADDED     = constraints in B but not A
CHANGED   = same local identifier, changed relation or expression
```

A changed constraint is not flattened into either simple loss or simple addition; the previous and subsequent forms remain attached as a transformation pair.

---

## 6. Fidelity is not identity

Define exact declared-registry equivalence:

```
Equivalent(A,B) iff Constraints(A) = Constraints(B)
```

This is intentionally narrow.

It does not assert:

```
Referent(A) = Referent(B)
```

nor:

```
A and B exhaustively mean the same thing.
```

It asserts only equality of the declared finite registry at the selected grain.

A weaker relation can be defined by an explicit bridge. For example, if a domain rule establishes that two expressions are interchangeable within a stated jurisdiction and date range, the registry may record:

```
BRIDGED_BY(A_constraint, B_constraint, rule_R)
```

The bridge itself carries provenance and scope.

---

## 7. Drift theorem schema

For finite declared constraint sets `CA` and `CB`:

```
if exists c such that c in CA and c not in CB,
then CA != CB.
```

Likewise:

```
if exists c such that c in CB and c not in CA,
then CA != CB.
```

This is mathematically trivial. Its usefulness lies in refusing to conceal the linguistic bridge that supplies membership in the sets.

The generated Lean receipt therefore has two layers:

### Bridge layer – not kernel-certified by the instrument

```
The analyst/parser declares that "knowingly" corresponds to
constraint mental_state in artifact A and does not occur as that
constraint in artifact B.
```

**Formal layer – Lean-ready**

```
"mental_state::REQUIRES::knowingly" belongs to sourceConstraints.
"mental_state::REQUIRES::knowingly" does not belong to targetConstraints.
Therefore sourceConstraints != targetConstraints.
```

The Lean theorem is only as semantically relevant as the bridge is warranted.

---

## 8. Provenance and authority

Every artifact and every bridge should carry, where available:

```
source identifier
source version
exact quotation / text
speaker or issuing body
recording/editor
jurisdiction
domain
authority class
date/effective date
supersession status
citation/location
parser version
analyst declaration
```

This matters especially in legal language. A dictionary definition, a style manual, a statute, a court opinion, a headnote, a brief, and a later AI summary do not have the same authority merely because they concern the same term.

The registry therefore treats authority as metadata on a relation, not as a magic truth bit.

**Authority-scoped findings**

A recognized adjudicating body may issue a finding that is binding, controlling, persuasive, advisory, superseded, or otherwise operative within a declared jurisdiction and time. MLFR records that fact as its own scoped relation:

```
AUTHORITY_FINDING(issuer, jurisdiction, time, proposition, status)
```

The registry does **not** silently rewrite that relation as:

```
ABSOLUTE_TRUTH(proposition)
```

Thus a court judgment can be represented as a binding or controlling finding within the court's recognized authority while remaining distinguishable from a claim that the proposition is frame-independently true. The same discipline

applies to dictionaries, standards bodies, scientific institutions, religious authorities, MetaLabe itself, and any other adjudicating registry.

The important distinction is:

```
recognized authority can settle an issue within its admitted scope
!=
recognized authority becomes an unscoped oracle of truth
```

This does not diminish jurisdiction. It states what jurisdiction is.

---

## 9. Etymology without etymological absolutism

Etymology is represented as a historical relation path:

```
current_form
  DERIVED_FROM earlier_form
  DERIVED_FROM root
```

It is not represented as:

```
root = timeless true meaning
```

The instrument may therefore show that a current term inherits a historical relation while also preserving a distinct current-use registry.

This is important for the broader MetaLabe work on nouns, nominalization, objectification, and frame loss. A portable lexical object can carry a history without re-instantiating that history on every use.

---

## 10. David-Wynn: Miller as a negative control, not oracle or target

David-Wynn: Miller's variously named "Correct Sentence Structure Communication Parse Syntax Grammar" / "Quantum Grammar" is useful here because it attempts an extreme solution to semantic and legal ambiguity: one privileged grammar is claimed to restore deterministic meaning.

This project neither adopts Miller's claims nor requires a total judgment of him. The divergence is methodological.

Miller sought, in effect:

```
one privileged grammar -> one correct parse -> determinate truth
```

MLFR instead records:

```
source + frame + parser set + domain rules + provenance
+ competing parses + bridges + residue + reopen conditions
```

The useful residue in Miller's practice is the insistence that punctuation, grammar, tense, definitions, formatting conventions, and document history can carry consequences that later users may forget. That observation is compatible with ordinary legal and linguistic practice. The unsupported move is converting those local conventions into a universally privileged private grammar.

The tool therefore uses Miller as a stress case: can the registry preserve the real local rule without inheriting the inflated conclusion?

---

## 11. Legal domain application

Legal language is a high-value test domain because textual changes that look small can alter scope, authority, agency, modality, exception structure, or burden.

The registry should distinguish at least:

```
primary law
secondary authority
style/editorial convention
party assertion
judicial holding
dictum
quotation
paraphrase
headnote
later summary
```

It should also support domain rules such as:

```
DEFINED_USING
REQUIRES
EXCEPTS
MODIFIES
SCOPES_OVER
PRESERVES
FORGETS
BRIDGED_BY
SUPERSEDED_BY
```

A legal dictionary can provide a useful lexical witness without becoming binding law. A style guide can certify an editorial convention without automatically determining substantive validity. A grammatical canon can support a construction without mechanically overriding context.

---

## 12. Why Lean belongs at the end of the pipeline

Lean's architecture cleanly separates parsing, elaboration, and kernel checking. MLFR uses that separation as a discipline.

Natural language is first mapped into a finite declared registry. The instrument then emits Lean source encoding that registry and finite relations among its atoms. Lean can later check the formal consequences.

The correct flow is:

```
natural language
      ↓
candidate observations
      ↓
analyst / domain bridge
      ↓
declared finite registry
      ↓
generated Lean source
      ↓
external Lean kernel run
      ↓
certified formal receipt
```

The forbidden shortcut is:

```
natural language -> "Lean proved the meaning"
```

No such claim is made.

---

## 13. Generated Lean receipt

The v0.1 instrument emits a self-contained Lean file using finite lists of strings. For each lost constraint it generates a theorem asserting membership in the source list and non-membership in the target list. For each added constraint it generates the converse. It also emits either:

```
sourceConstraints = targetConstraints
```

or:

```
sourceConstraints != targetConstraints
```

with a `by decide` proof candidate.

These are deliberately modest theorems. They are machine-checkable statements about the declared registry.

The file is labeled:

`LEAN-READY / NOT KERNEL-CERTIFIED BY THIS INSTRUMENT`

until an external Lean run supplies the actual receipt.

**Formalization map for this draft**

The accompanying generic kernel names the present proof obligations explicitly:

```
drift_witness_blocks_exact_equivalence
    finite registry drift blocks exact declared-registry equality

scoped_universal_seals_counterexample
    a proved universal excludes a same-carrier counterexample

double_edged_return
    a rule declared over a domain returns upon an adjudicator in that domain

denunciation_alone_is_not_refutation
    bare denunciation is not one of the admitted refutational challenge kinds
```

These generic theorem schemas are intended to be externally compiled and, after successful certification, attached to the paper as receipts. Domain-specific natural-language applications still require their bridge records.

---

## 14. Instrument self-check versus proof certification

The browser instrument performs deterministic internal checks:

- every preserved constraint must occur in both registries;
- every lost constraint must occur only in the source registry;
- every added constraint must occur only in the target registry;
- every changed constraint must share an identifier while differing in relation and/or expression;
- serialized receipt hashes are recomputed before export.

Passing these checks means:

`INSTRUMENT CONSISTENCY CHECK PASSED`

It does not mean:

`LEAN CERTIFIED`

The distinction is displayed in the interface and exported receipt.

---

## 15. Output receipt model

A receipt contains:

```
instrument version
analysis timestamp
raw artifact hashes
raw texts
artifact metadata
candidate observations
analyst-declared constraints
preserved constraints
lost constraints
added constraints
changed constraints
lexical delta
residue / notes
reopen conditions
instrument self-check result
Lean source
certification status = NOT_RUN
```

After a later Lean run, an external process may append:

```
Lean version
Mathlib version if used
command
exit status
stdout/stderr hash
#print axioms output
certification status
```

The browser does not fabricate those fields.

---

## 16. Pharmakon and the linguistic accumulator

The conceptual purpose is broader than legal comparison.

A compressed linguistic object can preserve enough state to answer the current query while failing to preserve enough state to update correctly when context changes.

Therefore:

```
presently adequate representation
!=
sufficient continuation state
```

This is the linguistic form of the constitutive pharmakon.

The cure is not refusal of compression. Communication requires compression.

The corrective discipline is:

```
carry enough basis to know what was preserved,
what was omitted,
what changed,
and what would reopen the conclusion.
```

---

## 17. Claim status

### Implemented in browser v0.1

- two-artifact comparison;
- provenance fields;
- deterministic candidate feature scan;
- lexical addition/removal scan;
- manual declared constraint registry;
- preserved/lost/added/changed classification;
- deterministic receipt construction;
- internal consistency checks;
- JSON receipt export;
- Lean-ready source generation and export;
- local operation without network dependency.

### Lean-ready but not kernel-certified here

- finite list equality/inequality receipt;
- per-constraint lost/added membership witnesses.

### Open / future

- full syntactic dependency parsing;
- multiple competing parse trees with parser provenance;
- jurisdiction-aware legal rule libraries;
- versioned Black's and historical legal dictionaries;
- GPO/editorial style-rule registry;
- citation-to-primary-authority graph;
- etymological relation graphs;
- translation alignment across languages;
- formal modal/deontic semantics;
- automatic Lean execution in an isolated certified environment;
- proof-carrying external receipt import;
- semantic lineage visualization across many generations of quotation/paraphrase.

---

## 18. Double-edged standard

An adjudicating criterion is not an equal standard if the adjudicator is exempt from the same criterion whenever the criterion would return an unfavorable result.

For an adjudicator `a`, domain `D`, and standard `J`:

```
DOUBLE_EDGED(J,D,a) requires a in D.

If the rule is declared to govern every x in D,
then the same rule governs a.
```

This does not imply that every authority has jurisdiction over itself in every institutional sense. It means that **when a claim presents a criterion as the reason an inference is warranted, the criterion cannot gain argumentative privilege merely because its application has returned to the claimant or adjudicator.**

The discipline is symmetric:

```
criterion applied outward
        |
        +---- same scope ----> criterion may return inward
```

A proposed exemption must therefore appear as an additional premise, jurisdictional rule, or scope distinction. It cannot remain hidden.

---

## 19. Scoped refutation and sealing

A proved or certified statement is **sealed only at its declared scope**. Sealing does not mean immune from criticism, socially unquestionable, or absolutely true. It means that once a proposition has been established from stated premises under stated rules, a same-scope refutation must identify an actual failure condition rather than merely restating nonacceptance.

The admissible challenge classes are:

```
PREMISE_FAILURE
SOURCE_OR_PROVENANCE_FAILURE
PARSE_FAILURE
BRIDGE_FAILURE
INFERENCE_FAILURE
SCOPE_MISMATCH
ADMISSIBLE_COUNTEREXAMPLE
FORMAL_ENCODING_FAILURE
KERNEL_OR_COMPILATION_FAILURE
```

A bare declaration such as:

```
"I reject this."
"That is wrong."
"An authority disagrees."
```

is not by itself a same-scope refutation. An authority's disagreement is instead recorded as an authority-scoped finding, and its legal or institutional effect is preserved according to that authority's jurisdiction. If the authority changes the operative scope, premise set, interpretation, or rule, that change is recorded explicitly.

For a universal formal statement:

```
forall x in D, P(x)
```

a same-domain counterexample would have the form:

```
x in D and not P(x).
```

Once the universal statement is proved under the admitted formalization, such a counterexample is excluded **within that formalization**. A valid critique must therefore challenge the proof, the membership claim, the bridge that placed the case in `D`, the interpretation of `P`, or the adequacy of the formalization to the external target.

This is the sealing rule inherited from the Rule 30 and Pharmakon work:

```
proof closes the declared formal branch;
it does not close every possible redescriptive branch.
```

---

## 20. Return / revision rule

A generated receipt may be challenged by naming:

```
incorrect source text
incorrect provenance
incorrect parser observation
incorrect promoted constraint
incorrect domain rule
invalid bridge
omitted materially contrary parse
incorrect formal encoding
failed Lean compilation
failed theorem
```

A later correction changes the registry and produces a new versioned receipt. It does not silently rewrite the historical receipt. A successful scoped challenge reopens precisely the dependency it defeats; it does not erase unrelated certified relations.

---

## 21. Conclusion

The instrument does not attempt to produce a language from which ambiguity has been abolished. It attempts to make semantic transformations inspectable.

Its governing principle is:

```
Do not freeze the frame.
Carry the frame.
```

And its formal continuity criterion is:

```
A later statement does not inherit the meaning of an earlier
statement merely because it shares words, referents, subject matter,
or intended conclusion.

Continuity is certified only relative to declared relations that the
transformation preserves.

Any omitted dependency, changed scope, altered referent, changed
modality, changed authority, changed registry, or unresolved parse
remains part of the residue.

A claim of equivalence must either preserve that structure or supply
an explicit bridge explaining the change.
```

This is not a solver that announces final meaning. It is an instrument for preserving the receipts by which meaning is carried, changed, lost, recovered, and contested.

---

## References and source classes for the draft


- MetaLabe / Thoth's Tool internal working corpus, especially the frozen-artifact and relation-registry specifications.
- U.S. Government Publishing Office, *GPO Style Manual* (editorial and punctuation conventions).
- *Facebook, Inc. v. Duguid*, 592 U.S. 395 (2021) (grammatical canons and contextual limits).
- Lean Reference Manual, sections on parsing, elaboration, and kernel checking.
- Legal-research guidance treating legal dictionaries such as Black's as useful secondary sources rather than binding law.
- Academic analyses of sovereign-citizen / pseudolaw language, including work discussing Miller's claimed deterministic grammar.


**Draft status:** Work in progress. Historical, linguistic, and legal bridges remain independently auditable and revisable. Formal Lean source generated by the

accompanying instrument remains pre-certification until run in an external Lean environment.